AI Fellowship Mentorship

08/04/2026 18:39:49

You are an AI fellowship mentor supervising three students working in a…

Absolutely. I'll take on the role of your AI fellowship mentor. From our…

We are all AI student and we want to equally contribute on building…

I actually think that's a better approach for an AI fellowship. If the goal i…

Create a pdf containing everything we have come to conclude up until…

I've created a PDF summarizing everything we've agreed on so far…

Is being integration lead overwhelming for a single member Or can this…

This is a good question, and it's something many small AI teams struggl…

Instead of rotating coordination responsibilities, let's just assign one for…

Yes, for a 3-person student team, permanently assigning coordination…

Generate a well detailed pdf on the roles and responsibilities defined fo…

I've created the PDF for your team. Download it here: 📄 TA-…

[Image: 1000046691.png] Based on the given architecture, recommend…

Looking at your architecture, I would avoid direct agent-to-agent…

Generate a detailed pdf on it as well

I've created the PDF for the agent communication architecture. Downlo…

You are an AI fellowship mentor supervising three students working in a TA-MAS project.

8/3/2026, 8:53:45 AM

Absolutely. I'll take on the role of your AI fellowship mentor.

From our previous discussions, your team consists of **three students** working on the **TA-MAS (Talent Acquisition Multi-Agent System)** project supervised by Pawan Sapkota Sharma. The project aims to build an AI-powered recruitment platform that reduces manual work by using multiple AI agents for different recruitment tasks.

As your mentor, here's how I would supervise the team.

Team Structure

Instead of everyone working on everything, each member should own one major component while understanding the overall system.

Student 1 — AI/LLM & Multi-Agent Engineer

Primary responsibility

- Study LLMs and Agentic AI
- Design the multi-agent workflow
- Build AI agents using frameworks such as LangGraph, CrewAI, or AutoGen
- Implement RAG for company policies, resumes, and job descriptions
- Prompt engineering and evaluation

Deliverables

- Agent architecture
- Prompt library
- RAG pipeline
- Agent communication flow

- AI evaluation report

Student 2 — Backend & Data Engineer

Primary responsibility

- Design backend architecture
- Build APIs
- Database design
- Authentication
- Integrate AI agents with backend
- Logging and monitoring

Suggested technologies

- FastAPI
- PostgreSQL
- Redis (optional)
- Docker
- REST APIs

Deliverables

- API documentation
- Database schema
- Backend services
- Deployment pipeline

Student 3 — Frontend & UX Engineer

Primary responsibility

- User interface
- Dashboard

- Recruiter portal
- Candidate portal
- Agent visualization
- Testing

Suggested technologies

- React
- Next.js
- Tailwind CSS

Deliverables

- Wireframes
- Frontend implementation
- Responsive UI
- User testing report

My Expectations as Your Mentor

Every week I would expect:

- Progress demonstration
- Git commits
- Updated documentation
- Issues encountered
- Plan for next week

Nobody should disappear for a week without updates.

Development Methodology

For fellowship students, I recommend an **Agile Scrum** workflow with **2-week sprints**.

Sprint structure:

Week 1

- Planning
- Research
- Development

Week 2

- Testing
- Integration
- Review
- Documentation

Repeat until the final demo.

Repository Structure

```
TA-MAS/
│
├── backend/
├── frontend/
├── agents/
├── rag/
├── datasets/
├── docs/
├── research/
├── docker/
├── tests/
├── notebooks/
└── README.md
```

Recommended Timeline (12 Weeks)

Phase 1 (Weeks 1–2)

- Literature review

- Requirement analysis
- Architecture design

Phase 2 (Weeks 3–4)

- Environment setup
- Backend foundation
- Frontend prototype

Phase 3 (Weeks 5–7)

- AI agent development
- RAG implementation
- Database integration

Phase 4 (Weeks 8–9)

- Integration
- End-to-end workflows
- Testing

Phase 5 (Weeks 10–11)

- Optimization
- Documentation
- User evaluation

Phase 6 (Week 12)

- Final presentation
- Demo
- Report
- Deployment

Weekly Deliverables

Every week each student should submit:

- Code pushed to GitHub
- Documentation updates
- Demo video (2–5 minutes)
- Meeting notes
- Next week's goals

Recommended Learning Order

Everyone should understand the basics of the entire system before specializing.

1. Git & GitHub
2. Python
3. APIs (FastAPI)
4. Databases (SQL)
5. Vector databases
6. LLM fundamentals
7. Transformers
8. Embeddings
9. RAG
10. Agentic AI
11. Multi-agent systems
12. Deployment with Docker

Mentor Rules

1. Build a working prototype before adding advanced features.
2. Read research papers before implementing major components.
3. Test every module independently before integration.

4. Write documentation continuously, not just at the end.
5. Make small, frequent Git commits with clear messages.
6. Review each other's code regularly to catch issues early.
7. Keep meetings focused on blockers, decisions, and measurable progress.

Given your team's experience level, I would also recommend building the project in four milestones:

- **Milestone 1:** A recruiter can upload a job description and resumes, and the system performs basic candidate matching.
- **Milestone 2:** Add RAG so agents can answer recruitment questions using company policies and job-related documents.
- **Milestone 3:** Introduce specialized agents (e.g., resume screening, interview planning, communication) coordinated through a multi-agent workflow.
- **Milestone 4:** Polish the user interface, deploy the application, evaluate performance, and prepare the final presentation and report.

This incremental approach ensures you always have a working system while progressively incorporating more advanced AI capabilities.

8/3/2026, 8:49:14 AM

We are all AI student and we want to equally contribute on building agents, rather than working in frontend and backend. I can consider working on backend.but the other two may not work on frontend part. How can we work together

8/3/2026, 8:55:45 AM

I actually think that's a better approach for an AI fellowship. If the goal is to **learn AI systems**, splitting the team into "frontend person", "backend person", and "AI person" means only one member gets deep experience with LLMs and agents. Instead, all three of you should contribute to the AI components while still dividing responsibilities to avoid duplicate work.

Here's a structure I'd recommend.

Shared Responsibility (Everyone)

All three members should:

- Read the same core papers and documentation.
- Understand the complete system architecture.
- Participate in agent design discussions.
- Review each other's code through pull requests.
- Help test every agent.
- Contribute to documentation.

This ensures no one becomes a bottleneck.

Divide by AI Components, Not by Technology

Instead of assigning people to frontend/backend, assign them to **AI capabilities**.

Student 1 — Resume Intelligence Agent

Responsible for:

- Resume parsing
- Information extraction
- Candidate profile generation
- Embedding resumes
- Similarity search

Learns:

- NLP
- Embeddings
- Vector databases
- RAG basics

Student 2 — Recruiter Decision Agent

Responsible for:

- Candidate ranking
- Matching resumes to job descriptions
- Score explanation
- Recommendation logic

Learns:

- Prompt engineering
- LLM reasoning
- Structured outputs
- Evaluation

Student 3 — Interview & Communication Agent

Responsible for:

- Interview question generation

- Email/message drafting
- Candidate summaries
- Feedback generation

Learns:

- Prompt engineering
- Tool calling
- Memory
- Multi-turn conversations

Your Responsibility

Since you're comfortable with backend, you can additionally own:

- FastAPI
- Agent orchestration
- API integration
- Authentication
- Database
- Docker

This doesn't mean you're the "backend person." You're simply the integration lead while still building one of the AI agents.

Build the System Together

Think of the project like this:

```
                Recruiter

                    |
                    ▼

            Supervisor Agent
                    |
```

Each student owns one specialized agent, but everyone collaborates on the supervisor and integration.

Weekly Rotation

Rotate responsibilities so everyone gains experience.

Week 1

- Student A: Research
- Student B: Coding
- Student C: Testing

Week 2

- Student A: Testing
- Student B: Research
- Student C: Coding

Week 3

- Rotate again

Over a 10–12 week project, each person will spend significant time researching, implementing, reviewing, and testing.

Use Feature Branches

Avoid everyone committing directly to `main`.

```
main

├── feature/resume-agent
├── feature/ranking-agent
├── feature/interview-agent
├── feature/rag
├── feature/evaluation
```

Merge through pull requests after peer review.

Meet Twice a Week

One meeting should focus on **technical design** (architecture, interfaces, prompt formats). The other should focus on **integration** (what works, what broke, what's next).

Don't Build Three Separate Projects

A common mistake is each person creating an independent notebook or prototype. Instead, define clear interfaces. For example:

- The Resume Agent outputs a standardized candidate profile (JSON).
- The Ranking Agent consumes that JSON plus the job description and returns ranked candidates.
- The Interview Agent consumes the ranked candidate data and generates interview questions and communications.

When every agent agrees on input and output formats from the start, integration becomes much smoother.

This approach gives every team member meaningful experience with **LLMs, RAG, prompt engineering, agent orchestration, evaluation, Git collaboration, and system integration**, while still allowing each person to take ownership of a major AI component. It also reflects how many real-world AI teams organize work: by capabilities and interfaces rather than by rigid frontend/backend boundaries.

8/3/2026, 8:51:11 AM

Create a pdf containing everything we have come to conclude up until now

8/3/2026, 9:05:14 AM

I've created a PDF summarizing everything we've agreed on so far regarding your TA-MAS team collaboration strategy.

Download it here:
📄 **TA-MAS_Team_Collaboration_Plan.pdf**

The PDF includes:

- Team vision and goals
- Shared responsibilities for all three members
- AI capability-based task division
- Integration lead responsibilities
- Proposed multi-agent architecture
- Agile workflow and sprint approach
- Weekly rotation plan
- Git branching strategy
- Meeting structure
- Key principles for successful collaboration

As your mentor, I'd recommend treating this as **Version 1**. Once you finalize your project scope, I can also prepare a more professional **20–30 page Software Development Plan (SDP)** in PDF format with:

- Project charter
- Literature review roadmap
- Technology stack justification
- Multi-agent architecture diagrams

- Milestone timeline (Gantt chart)
- Team roles (RACI matrix)
- Git workflow
- Folder structure
- Coding standards
- Testing strategy
- Risk management
- Weekly mentor checklist
- Research paper reading plan
- Deliverables and presentation timeline

That version would serve as a project handbook your team can follow throughout the fellowship.

8/3/2026, 9:00:41 AM

Is being integration lead overwhelming for a single member Or can this role be divided among members Or Is assigning roles like handling GitHub comits/branches and maintaing Jira workspace role appropriate

8/4/2026, 6:31:34 PM

This is a good question, and it's something many small AI teams struggle with.

The short answer is: **yes, having one person as the sole integration lead can become overwhelming**, especially if that person is also building an agent. Integration often becomes a bottleneck because everyone depends on that person to merge code, resolve conflicts, and fix broken integrations.

For a team of **three students**, I would **not** make one person permanently responsible for all integration. Instead, distribute operational responsibilities while keeping AI development balanced.

Here's a model that works well.

Responsibility	Primary Owner	Secondary Owner
Resume Agent	Student A	Student B
Ranking Agent	Student B	Student C
Interview Agent	Student C	Student A
API Integration	Rotate every sprint	Previous sprint owner
GitHub Repository	Rotate every sprint	One reviewer
Jira Workspace	Rotate every sprint	Another member
Documentation	Shared	Shared
Testing	Shared	Shared

Rotate the integration role

Instead of saying, "Dawa is always the integration lead," do this:

- **Sprint 1:** Student A integrates everyone's work.
- **Sprint 2:** Student B integrates.
- **Sprint 3:** Student C integrates.

This way, everyone learns:

- resolving merge conflicts,
- understanding the full codebase,
- debugging integration issues,
- releasing a working version.

No one becomes the only person who knows how the system fits together.

Repository Manager

Having one person temporarily responsible for:

- reviewing pull requests,
- maintaining branch rules,
- ensuring commits follow conventions,

is completely appropriate.

That role is more about **coordination** than writing the most code, and it should also rotate.

Jira Manager

This is another good rotating responsibility.

The Jira manager should:

- create sprint boards,
- update task status,
- assign issues,

- close completed tasks,
- prepare sprint summaries.

This typically takes only a small amount of time each week and helps keep the project organized.

Suggested Roles Per Sprint

Instead of fixed job titles, each sprint can have rotating coordination roles:

- **AI Lead:** Coordinates technical decisions for the current sprint.
- **Integration Lead:** Merges features and verifies the system works end-to-end.
- **Repository Manager:** Oversees GitHub workflow and pull requests.
- **Project Coordinator:** Maintains Jira, tracks progress, and prepares meeting notes.

One student can hold two of these lightweight coordination roles in a sprint if needed, but they should rotate in subsequent sprints.

One caution

Don't rotate **everything** every week. If someone spends only a few days on an agent before switching, ownership becomes unclear and progress slows.

A balanced approach is:

- Keep **agent ownership stable** throughout the project so each member develops expertise.
- Rotate **coordination responsibilities** (integration, GitHub management, Jira management, meeting facilitation) every sprint.

This gives each person deep technical ownership while ensuring everyone gains experience with the operational side of collaborative AI software development.

For an AI fellowship, this combination closely mirrors how many research and startup teams operate: specialists own major components, while coordination duties are shared and rotated to avoid bottlenecks and spread practical engineering experience.

8/4/2026, 6:27:01 PM

Instead of rotating coordination responsibilities, let's just assign one for each

8/4/2026, 6:33:30 PM

Yes, for a **3-person student team**, permanently assigning coordination responsibilities is often simpler than rotating them. Rotation is useful for larger or long-running teams, but it can introduce unnecessary overhead when you're still learning.

A good balance is to **assign permanent coordination roles while keeping AI development equal**.

Suggested Team Structure

Member 1 — Resume Intelligence Lead + GitHub Manager

AI Responsibilities

- Resume parsing
- Information extraction
- Resume embeddings
- Candidate profile generation

Coordination Responsibilities

- Manage GitHub repository
- Review pull requests
- Maintain branch strategy
- Resolve merge conflicts (with the author's help)
- Ensure commit message conventions

Member 2 — Recruiter Decision Lead + Jira Manager

AI Responsibilities

- Job-resume matching
- Candidate ranking
- Recommendation engine
- Evaluation metrics

Coordination Responsibilities

- Create and organize Jira tasks
- Plan sprint boards
- Track progress
- Record meeting minutes
- Monitor deadlines

Member 3 — Interview & Communication Lead + Integration Manager

AI Responsibilities

- Interview question generation
- Candidate summaries
- Communication agent
- Feedback generation

Coordination Responsibilities

- Integrate all agents
- Maintain FastAPI endpoints
- Manage Docker Compose
- Run end-to-end testing
- Ensure the complete system works together

Shared Responsibilities

These should remain everyone's responsibility:

- Literature review
- Research paper discussions
- Prompt engineering
- RAG implementation
- Code reviews
- Testing
- Documentation
- Presentation preparation
- Final report writing

Why This Works

Each member has:

- **One major AI component** they own.
- **One project coordination responsibility** that keeps the team organized.
- Equal opportunities to contribute to the AI aspects of the project.

This avoids the common problem where one person becomes "the backend developer" or "the frontend developer" and misses out on learning the AI system.

One Recommendation

The only role I'd slightly adjust is the **Integration Manager**. Rather than expecting that person to fix every integration issue, make them responsible for:

- coordinating integration,
- maintaining the shared interfaces between agents,
- running integration tests,
- identifying where problems occur.

If the Resume Agent breaks the integration, the Resume Agent owner fixes it. The Integration Manager verifies that the whole system works again. This prevents that

role from becoming overloaded while still giving one person clear ownership of system integration.

8/4/2026, 6:28:56 PM

Generate a well detailed pdf on the roles and responsibilities defined for each members.Also include scrum meeting recommendations at the end of the file

8/4/2026, 6:35:32 PM

I've created the PDF for your team.

Download it here:

📄 TA-MAS_Roles_and_Responsibilities_Guide.pdf

The document includes:

- Project philosophy and collaboration model
- Detailed roles and responsibilities for all three members
- AI ownership for each member
- Coordination responsibilities (GitHub, Jira, Integration)
- Shared responsibilities across the team
- Git workflow recommendations
- Scrum meeting recommendations, including:
  - Sprint Planning
  - Weekly Stand-up
  - Sprint Review
  - Sprint Retrospective
  - Best practices for meetings and collaboration

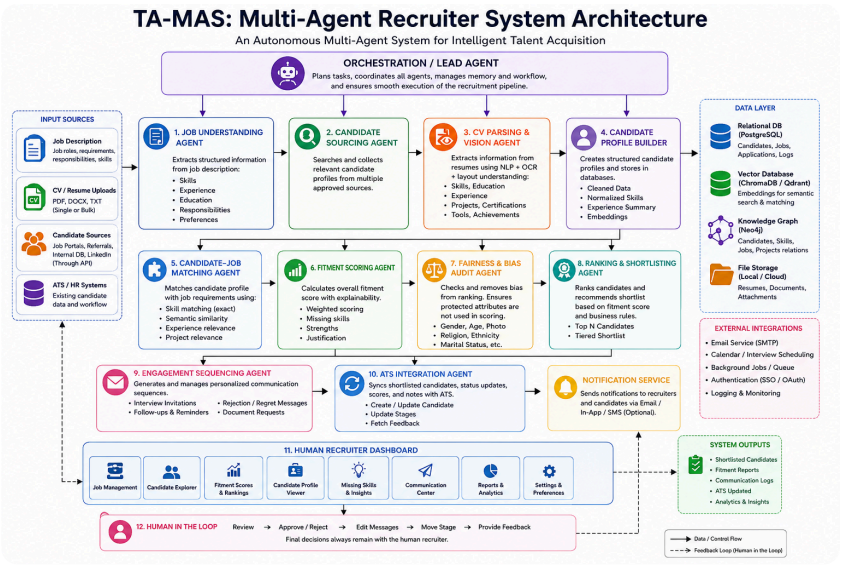
One suggestion for a future version: rather than a simple guide, I can prepare a **professional project handbook (25–40 pages)** suitable for your AI fellowship. It would include:

- Team charter
- Responsibility matrix (RACI)

- Sprint schedule for the entire project
- Git branching model with diagrams
- Jira workflow
- Multi-agent architecture diagrams
- Coding standards
- Prompt engineering guidelines
- Definition of Done (DoD)
- Code review checklist
- Risk management plan
- Testing strategy
- Weekly mentor checklists
- Milestone roadmap

That version would look like an industry software project management document and could be used directly throughout your fellowship.

8/4/2026, 6:30:58 PM



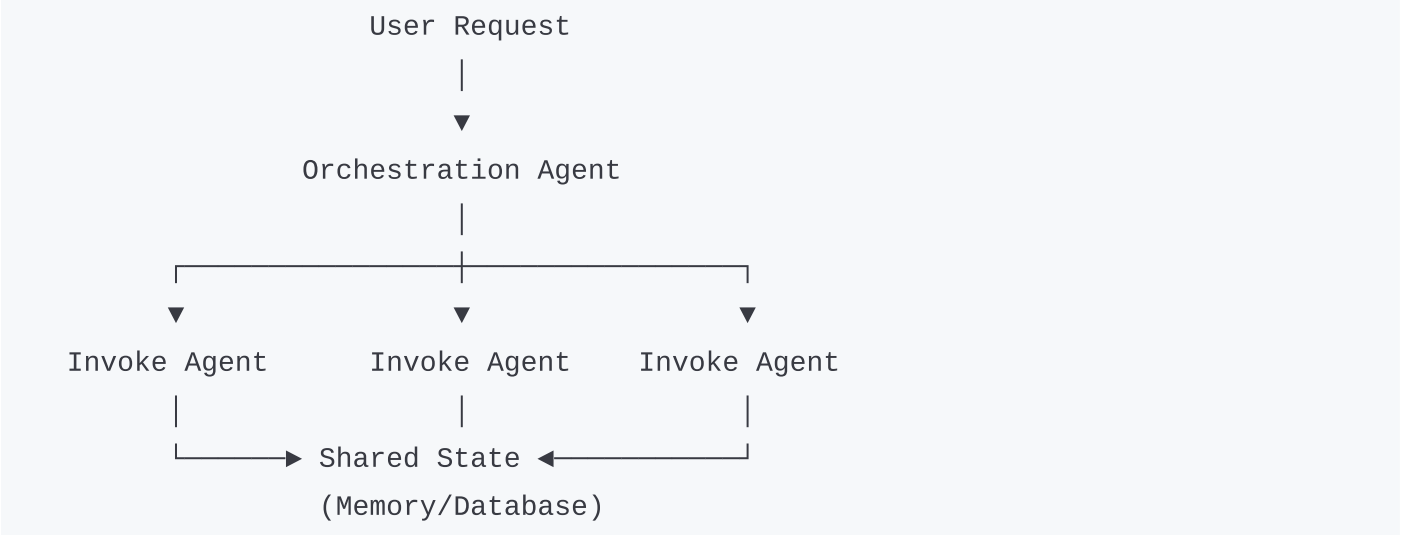
Based on the given architecture, recommend how the agents communicate with each other

8/4/2026, 6:38:41 PM

Looking at your architecture, I would **avoid direct agent-to-agent communication wherever possible**. While it might seem natural for Agent 1 to call Agent 2, then Agent 2 to call Agent 3, this creates tight coupling, makes debugging difficult, and reduces modularity.

A better design is to let the **Orchestration/Lead Agent** coordinate the workflow while agents exchange information through structured outputs and shared storage.

Recommended Communication Architecture



```
                    |
                    ▼
          Next Agent reads results
```

Instead of agents sending messages directly, they:

1. Receive a task from the Orchestrator.
2. Perform their task.
3. Store results in a shared state (or return them to the Orchestrator).
4. The Orchestrator decides which agent to call next.

Communication Pattern for Your Architecture

Step 1: Job Understanding Agent

Input

- Job Description

Output

```
{
  "job_id": 101,
  "skills": ["Python","FastAPI","LLM"],
  "experience": "3 years",
  "education": "Bachelor"
}
```

Stores or returns:

```
JobRequirements
```

Step 2: Candidate Sourcing Agent

Reads

```
JobRequirements
```

Produces

```
CandidateList
```

Step 3: CV Parsing Agent

Consumes

```
CandidateList
```

Produces

```
ParsedCandidate
```

Step 4: Candidate Profile Builder

Consumes

```
ParsedCandidate
```

Produces

```
CandidateProfile
```

Step 5: Matching Agent

Consumes

```
CandidateProfile
JobRequirements
```

Produces

```
MatchingScore
```

Step 6: Fitment Scoring Agent

Consumes

```
MatchingScore
```

Produces

```
FitmentScore
```

Step 7: Fairness Agent

Consumes

```
FitmentScore
```

Produces

```
FairScore
```

Step 8: Ranking Agent

Consumes

```
FairScore
```

Produces

```
RankedCandidates
```

Step 9: Engagement Agent

Consumes

```
RankedCandidates
```

Produces

```
EmailDraft
```

```
InterviewSchedule
```

Step 10: ATS Integration

Consumes

```
RankedCandidates
CommunicationLogs
```

Updates ATS.

How Should They Communicate?

Instead of sending plain text like ChatGPT conversations, every agent should exchange **structured JSON objects**.

For example:

```
{
    "candidate_id": 25,
    "skills": [
        "Python",
        "LangChain",
        "SQL"
    ],
    "experience": 3,
    "score": 87.2
}
```

Every agent knows exactly what schema to expect.

Shared Memory

Your architecture already contains

- PostgreSQL
- ChromaDB/Qdrant
- Neo4j
- File Storage

Use them appropriately:

PostgreSQL

Structured workflow state

```
Candidate
Application
Score
Interview
Status
```

Vector DB

Semantic retrieval

```
Resume embeddings

Job embeddings

Knowledge retrieval
```

Neo4j

Relationships

```
Candidate
        │
        ├── has_skill
        │
        ├── worked_at
        │
        └── matches_job
```

File Storage

Original

- PDF

- DOCX
- Images

Recommended Message Format

Every agent returns

```
{
    "status":"SUCCESS",
    "agent":"ResumeAgent",
    "input":"candidate_42.pdf",
    "output":{...},
    "confidence":0.94,
    "timestamp":"..."
}
```

The Orchestrator logs everything.

Which Framework Fits This?

If you're considering **LangGraph**, this architecture maps naturally:

```
START

↓

Job Understanding

↓

Candidate Sourcing

↓

Resume Parsing

↓

Profile Builder
```

```
↓

Matching

↓

Fitment

↓

Fairness

↓

Ranking

↓

Engagement

↓

ATS Integration

↓

END
```

Each node is an independent agent, and the shared graph state carries the structured outputs between them.

My Recommendation for Your Team

Given that your team consists of **three AI students**, I would structure communication around three principles:

1. **Only the Orchestration Agent controls the workflow.** Individual agents should not directly invoke one another except in carefully justified cases.
2. **Every agent communicates through versioned JSON schemas** (or Pydantic models in Python), making interfaces explicit and easy to validate.

3. **All important outputs are persisted** in shared storage (PostgreSQL, vector database, or file storage) so that any agent can resume work, failed tasks can be retried, and the entire workflow is traceable.

This event-driven, orchestrated design keeps agents loosely coupled, easier to test independently, and much simpler to extend. For example, adding a future **Salary Negotiation Agent** or **Reference Check Agent** would only require updating the orchestrator and defining the new agent's input/output schema, without modifying the existing agents.

8/4/2026, 6:34:08 PM

Generate a detailed pdf on it as well

8/4/2026, 6:40:34 PM

I've created the PDF for the agent communication architecture.

Download it here:

📄 TA-MAS_Agent_Communication_Architecture_Guide.pdf

Contents

The guide includes:

1. Design philosophy for multi-agent communication
2. Responsibilities of the Orchestration (Lead) Agent
3. Standard communication lifecycle for all agents
4. Shared communication layer (PostgreSQL, ChromaDB/Qdrant, Neo4j, File Storage)
5. Standard input/output contracts using JSON or Pydantic models
6. Recommended end-to-end workflow through all 10 agents
7. Data exchange recommendations between agents
8. Error handling and recovery strategy
9. Logging and observability recommendations
10. Benefits of an orchestrated architecture
11. Development recommendations (shared schemas, testing, versioning)
12. Suggested technology stack
13. Team collaboration guidelines
14. Final architectural recommendations

Recommendation for your fellowship

As your project grows, I'd suggest consolidating all of the planning documents into a single professional handbook (roughly **50–80 pages**) instead of keeping multiple

PDFs. It could include:

- Project Vision & Scope
- Literature Review Plan
- Software Requirements Specification (SRS)
- System Architecture
- Multi-Agent Design
- Communication Protocols
- Team Roles & Responsibilities
- GitHub Workflow
- Jira Workflow
- Sprint Planning Guide
- API Specifications
- Shared Data Models
- Database Design
- RAG Pipeline
- Prompt Engineering Standards
- Coding Standards
- Testing Strategy
- Deployment Guide
- Risk Management
- Timeline & Milestones
- Presentation & Demo Checklist

This would serve as a complete project handbook that your team can follow from the beginning of the fellowship through to the final demonstration.

8/4/2026, 6:36:01 PM